

讓一個校園問題， 成為能用的網站。

今天不急著寫程式。先知道：我們要做什麼，怎麼知道做對了。

從需求到應用

認識網站 × AI 協作 × 你的第一個提案

下課前，你能回答這四個問題。

不需要今天就會所有語法；你需要先有一張開發地圖。

01

做給誰用？

說出一位使用者，以及他真正遇到的困擾。

02

網站怎麼運作？

分清楚瀏覽器、後端與資料庫的工作。

03

AI 怎麼幫忙？

把需求說清楚，並查證 AI 給你的結果。

04

怎樣才算完成？

用一條完整流程與具體測試，證明功能可用。

小安想參加活動， 卻不知道自己有沒有報名成功。

接下來用這個例子，走過一個網站從構想到驗收的過程。

校園情境

海報上只有表單連結。

填完後沒有查詢入口。

群組又有人問：「還有名額嗎？」

主辦人也很困擾

名單裡有人填了兩次。

取消報名只能私訊處理。

活動快開始，仍在手動核對名額。

同一場活動，學生需要「確定結果」，主辦人需要「可信的名單」。

如果只能改善一件事， 你會先解決什麼？

兩人討論：指出一個困擾，說明改善後會有什麼不同。

「讓頁面更漂亮」

還是

「讓小安確認有沒有報名成功」？

▼ 想一想，再展開比較

兩者都可能價值。但在這個情境，先完成「報名 → 得到結果 → 能查詢」，才直接回應小安的不確定。漂亮的畫面不能取代缺少的流程。

先跑起來，再做完整， 最後交給別人試用。

14 週教學 + 2 週專題報告 + 2 週自主學習。

01—04 週

看懂與起步

第 1 週認識網站

第 2 週安裝環境

第 3—4 週頁面與表單

05—09 週

做出核心流程

資料庫與資料操作

需求、整合與除錯

第 9 週期中報告

10—16 週

讓系統可信

權限、規則與測試

整合、執行與互測

第 16 週期末報告

17—18 週

自主學習

提供分類資源

自行補強與延伸

不另塞核心必修功能

我們不比程式碼多， 也不比誰的 AI 方案比較貴。

AI 可以協助；你仍需要對交出的成果負責。

5%

出席

依實際出席紀錄

35%

隨堂作業與考試

課堂練習與測驗
理解並完成指定任務

25%

期中報告

問題清楚
核心流程可運作

35%

期末報告

正確性、測試
與試用後的改善

分組完成作品，每個人仍要能說明自己的貢獻。

小安報名之後， 主辦人應該看到同一個結果。

先操作，再想一想：背後有哪些事情必須做對？

學生視角 · 小安

校園攝影散步

名額 3 位 · 已報名 0 位

尚未報名。試試連按兩次「小安報名」。

主辦人視角 · 名單

目前沒有報名紀錄。

觀察：重複點擊會不會多一筆？取消後名單有沒有同步更新？

教學互動模型：這頁用 JavaScript 在瀏覽器模擬，沒有 ASP.NET 後端、登入或資料庫；重新整理會重設名單。

不要只做「報名」按鈕。 要做出按下去之後的結果。

完整流程，就是使用者從開始到達成目的，中間沒有斷掉。



▼ 只有跳出「報名成功」，算完成嗎？

不一定。要確認紀錄真的保存、使用者能查到，而且主辦人看到的是同一筆有效資料。提示文字本身不是成功的證據。

畫面、規則、紀錄。 各自有不同的工作。

把這三個部分分清楚，比背下一段程式更早派上用場。

01

瀏覽器 / 前端

顯示活動與表單，接收小安的操作，
呈現後端回傳的結果。

02

後端 / ASP.NET

接收請求，檢查身分、名額與重複報名，
決定是否接受。

03

資料庫

保存活動、使用者與報名紀錄，提供
後續查詢。

瀏覽器 ↔ 後端 ↔ 資料庫；一般情況下，不讓瀏覽器直接拿資料庫帳密操作資料。

追蹤一次請求的旅程。

逐步點擊，注意哪一步是在畫面，哪一步是在後端。

01 / 瀏覽器

小安按下「報名」。

瀏覽器送出報名請求，帶上活動編號；已登入身分由後端驗證，不相信任意填入的姓名。



HTTP：瀏覽器與伺服器交換請求和回應的一套規則。

ASP.NET 是做網站的框架； C# 是本課用來寫後端的語言。

前後端可以有很多技術選擇；這門課先專注一條能完成應用的路。

瀏覽器裡

HTML · CSS · JavaScript

HTML：內容與結構

CSS：外觀與版面

JavaScript：瀏覽器裡的互動

伺服器端

.NET · C# · ASP.NET

.NET：開發平台

C#：本課使用的程式語言

ASP.NET：處理網站與 Web API 的框架

今天先理解角色。實際開發工具與框架版本，第二週統一設定。

「已報名 3 人」可能只是頁面文字。

判斷功能完成，要查行為與資料，不能只看截圖。

把數字寫死

```
<p>已報名 3 人</p>
```

每次顯示同一句。

不代表系統真的保存了三筆報名。

從有效紀錄算出來

讀取這場活動的報名紀錄

→ 計算有效人數

→ 顯示查詢結果

有人取消，查詢結果也應跟著改變。

▼ 按 F5 重新整理後，資料還在，就一定用了資料庫嗎？

也不一定。瀏覽器本機儲存也能保留資料，但不代表其他裝置或主辦人能取得同一份資料。共享且可信的結果，需要適當的後端與資料保存設計。

把「活動」和「報名」分開想。

現在先理解「一筆資料代表什麼」，第五週再動手設計資料庫。

活動資料

活動編號	名稱	名額
A01	校園攝影散步	3

報名紀錄

紀錄編號	活動編號	使用者編號
R01	A01	U01 (小安)

一場活動可以對應多筆報名。編號用來連結資料，不靠同名姓名判斷是不是同一人。

表格為教學用資料。正式設計還會考慮狀態、時間與唯一性限制。

功能名稱不是完整需求。

把小安的困擾，轉成能讓別人理解的句子。

太模糊

我要一個報名系統。

不知道誰使用、要做什麼，也不知道成功的標準。

可以開始討論

身為學生，我想查看自己的報名狀態，
以便確認是否取得名額。

使用者 + 要做的事 + 為什麼需要。

先說「想達成什麼」，再討論要幾個頁面、幾個按鈕。

一句「可以報名」， 其實包含好幾個判斷。

規則越明確，你、同學與 AI 才越容易對「完成」有相同理解。

→ 活動開放中、有名額，而且尚未報名 → 建立一筆紀錄。

→ 同一使用者再次報名同一活動 → 不新增，說明已報名。

→ 名額已滿 → 不新增，說明額滿。

→ 報名成功後 → 學生可查狀態，主辦人可查名單。

驗收條件：在什麼情況，做什麼操作，應該出現什麼結果。

「幫我做一個網站」 會留下太多空白讓 AI 猜。

AI 很會產生內容；你需要負責方向、限制與查證。

模糊要求

幫我做一個很厲害的
校園活動報名網站。

可能得到漂亮畫面，卻漏掉名額、重複報名與查詢。

具體要求

先做單一活動的報名流程。

說明使用者、資料與規則。

列出測試方式，再逐步實作。

一次完成一小段，確認後再往下走。

先請 AI 幫你釐清， 不要立刻產生整個專案。

今天可以先討論；有可用的 AI 工具再試，不需購買方案。

我要規劃校園活動報名網站，使用者是學生與主辦人。
先聚焦：學生報名一場活動，並確認自己的報名狀態。
限制：名額 3 人；同一使用者不可重複報名；額滿不得新增。

請先列出：

1. 還需要向我確認的問題。
2. 最小的頁面、資料與操作流程。
3. 正常、重複報名、額滿時的驗收條件。

先不要寫程式。不要自行加入付款、推薦或聊天功能。

按一次就加一人，哪裡不對？

假設 AI 給你類似的邏輯，你要靠什麼判斷它不合格？

故意有錯的互動模型

小安報名同一場活動

0 / 3 位

連按幾次，觀察人數。

不要只測一次成功。

同一人按第二次呢？

超過三次呢？

這個數字真的是不同使用者嗎？

▼ 展開問題

它只累加次數，沒有辨識使用者，也沒有檢查名額。按鈕會動，但不符合需求。真正後端還必須處理同時送出的請求，不能只靠停用按鈕。

此頁為故意保留錯誤的前端模型，不是 ASP.NET 實作，也不是實際 AI 服務輸出。

先寫預期、再執行，最後比較結果。

一張好看的成功截圖，無法證明其他情況也正確。

情況	操作	預期結果
尚有名額，小安未報名	小安報名	新增一筆；可查到成功狀態
小安已報名	小安再報一次	不新增；說明已報名
三個名額皆已使用	另一位學生報名	不新增；說明額滿
小安不是主辦人	嘗試修改活動名額	後端拒絕；資料不變

今天先學驗收思考；之後會把關鍵規則寫成可重複執行的測試。

小步完成，逐步確認。

不是一次生成整個網站，再祈禱它能跑。

01

描述需求

誰要做什麼？

02

拆小任務

先完成一條流程。

03

協作實作

AI 協助，你讀懂。

04

測試查證

實際是否符合預期？

05

修正再測

別一次改所有東西。

遇到錯誤，提供「做了什麼 / 預期什麼 / 實際發生什麼 / 錯誤訊息」。

不用背每一行， 但不能對每一行都沒概念。

例如：不用今天默寫登入程式，但要知道「藏起按鈕」不是完整權限控制。

01

看得懂

能追出資料從哪裡來、條件在哪裡判斷。

02

改得動

名額規則改變時，知道要檢查哪些地方。

03

驗得出

知道怎麼測，能分辨成功訊息與真正成功。

C#、資料模型、請求流程、權限與除錯，會跟著每週功能逐步練習。

工具可以借力， 理解與責任不能外包。

每個人都需要能說明自己的程式與判斷。

可以這樣用

- 請 AI 解釋概念、提供草稿。
- 協助除錯、補測試案例。
- 記錄一個查證或修正的例子。
- 說明自己採用、修改了什麼。

這些事情不要做

- 交出完全無法解釋的成果。
- 貼入真實個資、密碼或金鑰。
- 把 AI 說「成功」當作測試結果。
- 把沒有做過的測試寫成完成。

使用測試資料。不要求購買特定 AI 方案；沒有帳號也能參與討論與紙上練習。

從你找得到的使用者開始， 不要從功能清單開始。

今天先寫假設；之後找實際使用者確認，不把猜測當成訪談結果。

先問三個問題

1. 誰最近遇過這個困擾？
2. 他現在怎麼處理？
3. 哪一步最麻煩或容易出錯？

小安的題目示範

使用者：參加活動的學生。

現在：填表後私訊主辦人。

困擾：無法立即確認報名結果。

改善：報名後能自行查詢狀態。

第一版不是整個校園平台。 先讓小安完成一次報名。

你的題目可以不同，但也要能用一句話說出核心流程。

第一版要有

一條走得完的流程

看活動 → 報名 → 查詢結果。

主辦人看得到正確名單。

保留必要的名額與重複檢查。

先不做

與核心問題無關的擴張

付款、聊天、點數、推薦。

把全校所有服務整合進來。

也不為了「有 AI」硬加聊天機器人。

普通題目也能做得很好。重要的是：需求清楚、流程正確、有人願意試用。

寫出你的第一張問題提案。

不需要寫程式，也不需要先使用 AI。先用自己的話說清楚。

誰會使用？

例：參加社團活動的學生

現在遇到什麼困擾？

例：填表後不知道有沒有取得名額

希望完成哪一條流程？

例：看活動 → 報名 → 查詢結果

怎麼知道做對了？

例：重複報名不新增，而且可查詢原結果

第一版先不做什麼？

例：付款、聊天、推薦

下一步要找誰確認需求？

寫角色即可，不填真實姓名或聯絡資料

可寫在紙上或填入此頁。表單不會上傳；本機儲存可用時會自動保留草稿，共用電腦請用下方「清除草稿」。

對方聽完，能說出 「誰、做什麼、怎麼算成功」嗎？

先幫彼此釐清，不急著評斷題目夠不夠厲害。

→ 有沒有明確使用者，而不只是「所有人」？

→ 困擾是真正的問題，還是只有一個功能名稱？

→ 流程能從開始走到結果嗎？

→ 驗收條件能實際操作檢查嗎？

給對方一個具體問題：「如果同一人按兩次，你希望系統怎麼處理？」

四個問題，檢查你的開發地圖。

先自己回答，再展開答案；答不出來的地方，就是下次要補強的地方。

▼ 01 把管理按鈕藏起來，就能防止一般學生修改名額嗎？

不行。後端必須驗證權限，不能只依靠畫面有沒有按鈕。

▼ 02 畫面顯示「成功」，還需要檢查什麼？

紀錄是否保存、資料是否正確，以及學生與主辦人是否能查到符合權限的結果。

▼ 03 AI 給你一個看起來能跑的網站，下一步是什麼？

對照需求，閱讀關鍵邏輯，實測正常與例外情境，再修正並重新驗證。

▼ 04 今天不會寫完整 C# 程式，也能完成什麼？

說明使用者與困擾、畫出流程、辨識前後端與資料庫，以及寫出驗收條件。

第二週，我們會親手 跑起第一個 ASP.NET 網站。

下週的完成標準：建立 → 啟動 → 修改首頁 → 停止 → 再次啟動。

01

帶好設備

筆電、充電器，確認可用空間與網路；使用教室電腦者確認可用設備。

02

確認安裝權限

知道電腦是否允許安裝軟體；密碼由本人輸入，不交給 AI 或同學。

03

先回報限制

Windows / macOS、無法安裝、無筆電等情況，提前告知以安排。

本週先不用自行猜版本安裝。依第二週統一清單操作，減少環境差異。

帶著今天的問題，去找答案。

自主閱讀：找一個你今天還不熟的詞，用自己的話解釋。

MDN · 網站如何運作

對照瀏覽器、伺服器與 HTTP。

https://developer.mozilla.org/zh-TW/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works

Microsoft Learn · C# 概觀

先理解語言能做什麼，不需背語法。

<https://learn.microsoft.com/zh-tw/dotnet/csharp/tour-of-csharp/overview>

Microsoft · ASP.NET

認識框架用途；版本與工具下週統一。

<https://dotnet.microsoft.com/zh-tw/apps/aspnet>

連結需網路；簡報本身與頁內互動不需要網路。官方文件的章節與網址可能隨時間調整。

AI 可以幫你寫。 你要知道為誰而寫， 也要證明它寫對了。

使用者

完整流程

驗收證據

今天的成果：一張問題提案。下週的成果：你電腦上第一個能執行的 ASP.NET 網站。
